

平时作业汇总

- 信管2301
- 王荣崇
- 202321054020

第一次作业

数据挖掘与机器学习

第一次作业

信管2301

王荣崇

2021321054020

《白话机器学习算法》

1.1 准备数据

原则：“垃圾进，垃圾出”（Garbage In, Garbage Out）。高质量的数据是成功分析的前提。

1.1.1 数据格式

最常用格式：表格。

行 (Row)：代表一个数据点/观测结果。

列 (Column)：代表一个变量/特征/属性。

1.1.2 变量类型

二值变量 (Binary)：只有两种取值（如：是/否，买/不买）。

分类变量 (Categorical)：有多个离散类别（如：顾客类别、颜色）。

整型变量 (Integer)：用整数表示的数值（如：购买数量）。

连续变量 (Continuous)：用小数表示的精细数值（如：支出金额、温度）。

1.1.3 变量选择

目的：从原始数据的众多变量中，筛选出与研究目标最相关的变量。

原因：避免“维度灾难”，减少计算负担和噪声干扰，提升模型性能。

方法：通常是一个试错过程，可借助相关性分析等初步筛选。

- **1.1.4 特征工程**

定义：对原始变量进行创造性处理，以生成更有用的新特征。

例子：

重新编码：将具体的“顾客类别”归纳为更广义的“食草/食肉动物”。

降维：合并多个相关变量（如“膳食纤维”+“维生素C”）形成一个新主成分。

- **1.1.5 缺失数据**

处理方法：

近似 (Imputation)：

分类/二值变量：用众数（出现最频繁的值）填充。

数值变量：用中位数填充。

计算 (Prediction)：使用监督学习算法，基于其他特征来预测缺失值（更准确但更耗时）。

移除 (Removal)：万不得已时，删除包含缺失值的整行数据（会损失信息，可能导致样本偏差）。

1.2 选择算法

核心：根据任务目标选择合适的算法。

1.2.1 无监督学习

目标：发现数据中隐藏的模式或结构（我们事先不知道模式是什么）。

典型任务：聚类（分组）、降维、关联规则挖掘。

例子：根据购物记录对顾客分群；找出经常被一起购买的商品组合。

1.2.2 监督学习

目标：利用已知的输入-输出模式，对新数据进行预测。

前提：需要有带标签的历史数据（即已知正确答案的数据）。

两大问题：

回归 (Regression)：预测连续值（如房价、销售额）。

分类 (Classification)：预测离散类别（如是否买鱼、邮件是否为垃圾邮件）。

1.2.3 强化学习

目标：通过与环境的交互和反馈（奖励/惩罚）来学习最优策略，并能持续改进。

特点：模型在部署后仍能根据新反馈进行自我优化。

例子：在线广告投放系统根据点击率动态调整广告展示策略。

1.2.4 注意事项

选择算法还需考虑数据类型、结果解释性、计算复杂度等因素。

1.3 参数调优

比喻：像调节收音机频道一样，找到最佳设置。

目的：调整算法的内部设置（参数），以优化模型性能。

常见问题：

过拟合 (Overfitting)：模型过于复杂，完美拟合了当前训练数据（包括噪声），但对新数据预测效果差（泛化能力弱）。

欠拟合 (Underfitting)：模型过于简单，未能捕捉到数据中的基本模式，对当前和新数据的预测都不准。

解决方案：

正则化 (Regularization)：引入惩罚项，人为增大对模型复杂度的惩罚，防止过拟合，使模型更简单、泛化能力更强。

1.4 评价模型

核心：不能只看模型在训练数据上的表现，必须评估其对新数据的预测能力。

1.4.1 分类指标

预测准确率：正确预测的比例。简单但可能掩盖问题（如不平衡数据集）。

混淆矩阵：详细展示预测结果的四种情况（真正、假正、真负、假负），能揭示模型在哪类错误上更多。

1.4.2 回归指标

均方根误差：对预测误差取平方后再开方。它会放大较大误差的影响，对异常值敏感。

1.4.3 验证

目的：评估模型在未见过的数据上的表现。

方法：

训练集/测试集划分：将数据随机分为两部分，一部分训练模型，另一部分测试模型。

交叉验证：当数据量较小时使用。将数据分成 k 份，轮流用其中 $k-1$ 份训练，1份测试，最终取 k 次结果的平均值。这种方法能更可靠地评估模型性能。

1.5 小结

数据科学研究的四个关键步骤环环相扣：

准备数据：确保输入质量。

选择算法：匹配任务需求。

参数调优：优化模型性能，平衡拟合与泛化。

评价模型：通过严谨的验证方法，客观衡量模型的真实价值。

本节概念汇总

1. 基础概念

大数据：

定义：指体量巨大、类型繁多、价值密度低、处理速度快的海量数据集合。

4V特征：

Volume (体量巨大)：数据规模庞大。

Variety (类型繁多)：包含结构化、半结构化、非结构化等多种数据形式。

Value (价值密度低)：有价值的信息隐藏在大量数据中，需要挖掘。

Velocity (处理速度快)：要求快速采集、处理和分析数据。

数据挖掘：

定义：从大量的、不完全的、有噪声的、模糊的、随机的实际应用数据中，提取隐含在其中的、人们事先不知道的、但又是潜在有用的信息和知识的过程。

本节概念汇总

• 2. 数据与学习范式

数据类型：

结构化数据：如数据库表格，格式规整。

半结构化数据：如XML、JSON，有一定结构但不严格。

非结构化数据：如文本、图片、视频，无固定格式。

学习范式：

有监督学习：训练数据带有“标签”（已知答案），目标是学习输入到输出的映射关系（如分类、回归）。

无监督学习：训练数据没有标签，目标是发现数据内部的结构或模式（如聚类、关联规则）。

本节概念汇总

3. 数据预处理技术

数据预处理 / 数据清洗：对原始数据进行清理、填补缺失值、纠正错误等操作，为后续分析做准备。

数据标准化 / 数据归一化：将不同量纲或取值范围的特征缩放到一个统一的标准尺度（如0-1之间），以消除量纲影响，使模型更稳定。

数据降维：减少数据集的变量（特征）数量，同时尽可能保留重要信息。常用方法包括主成分分析（PCA）、t-SNE等。

衍生变量：通过对原始变量进行计算、组合或转换而创建的新变量，旨在更好地揭示数据中的潜在规律。

第二次作业

1.数据回归方法的基本方法、步骤

数据回归方法是统计学中用于描述、解释和预测变量之间关系的核心工具，其流程遵循严谨的逻辑步骤，从问题界定到模型应用形成闭环，确保分析结果的科学性与有效性。

01. 界定问题与变量定义

明确核心研究问题，精准识别因变量（Y）与自变量（X），厘清变量间的逻辑关联，为后续分析奠定基础框架。

02. 样本数据的采集与预处理

系统收集相关样本数据，严格检查并处理缺失值、异常值，统一变量量纲，保证数据的完整性、准确性与一致性。

03. 变量间关系的初步探索

通过绘制散点图、计算相关系数等方式，直观观察并量化变量间的关联特征，判断是否存在潜在的线性或非线性相关趋势。

04. 回归模型选型

结合数据特征与研究目的，选择适配模型，如一元/多元线性回归用于连续变量预测，Logistic回归用于分类变量分析。

06. 模型效果评价

利用 R^2 、F检验、p值、残差分析等指标，从拟合优度、显著性、误差分布等维度全面评估模型的解释力与合理性。

05. 模型参数估计

运用最小二乘法、极大似然估计或损失函数最小化等数学方法，求解模型系数，确定变量间具体的数量依存关系。

07. 模型应用与验证

检验模型的适用性与稳健性，将通过验证的模型用于解释现实变量关系，或代入新样本进行预测，形成分析结论。

2.一元线性回归和多元线性回归的基本数学模型

01 一元线性回归模型

$$y = \beta_0 + \beta_1 x + \varepsilon$$

y为因变量，x为自变量， β_0 是截距， β_1 是回归系数， ε 是随机误差。 β_1 表示x每增加一个单位时，y的平均变化量，反映了自变量对因变量的影响强度。

02 多元线性回归模型

$$y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + \varepsilon$$

x_1 至 x_p 为多个自变量， β_1 至 β_p 为对应回归系数。 β_i 表示在控制其他变量不变的情况下，自变量 x_i 对因变量y的独立影响程度，是一元模型的自然扩展。

03 矩阵表示形式

$$Y = X\beta + \varepsilon$$

将变量和参数以矩阵形式整合，是多元线性回归的标准数学表达。该形式极大简化了计算过程，便于利用线性代数工具和计算机算法进行参数估计与模型求解。

核心逻辑总结：线性回归的本质是通过线性方程拟合自变量与因变量的关系，一元模型揭示单一因素影响，多元模型纳入多变量分析，矩阵形式则为大规模数据计算提供了高效的数学基础。

3.Logistic 回归的基本数学模型及含义

Logistic 回归是经典的**0/1二分类模型**，广泛应用于信贷审批（是否贷款）、金融趋势（股票涨跌）、用户运营（客户是否流失）等场景。其核心是通过自变量计算样本属于类别“1”的概率，再映射为分类结果。

01. 概率形式 (Sigmoid函数)

设 $p = P(Y=1|X)$ 为样本属于1类的概率，模型通过 Sigmoid函数将线性结果压缩至(0,1)区间：
$$p = 1 / [1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}]$$

02. 对数发生比形式 (线性可解)

对概率进行对数变换，将非线性模型转化为自变量的线性组合，便于参数求解：
$$\ln[p/(1-p)] = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p$$



关键概念解析：

$p/(1-p)$ 称为“发生比(优势比)”，表示事件发生与不发生的概率比值；其对数形式即为“对数发生比”，是Logistic回归可解释性的核心。



决策实现逻辑：

模型先输出连续的概率值，再通过设定阈值（通常取0.5）进行离散化判断：概率大于阈值预测为类别“1”，反之则为“0”。

4.Logistic 回归与普通线性回归的联系与区别

01 核心联系：同源的监督学习逻辑



同属监督学习范式

二者均利用已有标注数据构建模型，核心目标都是探索自变量与因变量之间的映射关系，遵循监督学习的基本逻辑框架。



共享模型构建核心逻辑

都需要通过数据估计模型系数，最终模型既可以用于解释自变量对因变量的影响机制，也能对新的未知样本进行预测分析。



支持多自变量的建模能力

均可纳入多个自变量进行分析，能处理复杂的多因素影响场景，是解决多元数据分析问题的常用线性建模工具。

02 关键区别：模型本质的核心差异

01 因变量与预测目标

线性回归：因变量为连续数值，直接预测具体数值结果。

Logistic回归：因变量为0/1分类变量，预测样本属于某类的概率。

02 输出范围与参数估计

线性回归：输出无固定边界，常用最小二乘法估计参数。

Logistic回归：输出被S形函数限制在0~1，用极大似然法估计。



03 系数含义与应用场景的本质差异

线性回归系数表示自变量每变化一个单位，因变量的平均变化量，直接反映数值影响程度；而Logistic回归系数反映自变量对“对数发生比”的影响，需通过转换才能解释概率变化。前者适用于数值预测（如房价、销量），后者适用于分类判断（如疾病诊断、用户流失预测）。

5. 《白话机器学习算法》第六章 “回归” 读书笔记



01 核心案例：房价预测

以房价预测为切入点，展示了从单一变量趋势线到多变量加权组合的演变逻辑。其中，**回归系数**是核心，它代表变量对结果的影响程度与方向：正则同向变化，负则反向变化；标准化后的系数可直接对比不同变量的重要性，让预测逻辑更清晰。



02 方法的优劣势分析

✓ **核心优势：**模型直观易懂，计算速度快，输出的回归系数能直接解释变量对结果的具体影响，便于业务人员理解和沟通。

✗ **主要局限：**对异常值高度敏感，易出现多重共线性问题，且仅适用于变量间存在线性关系的数据场景。



03 关键认知与思考

回归分析不仅用于预测，更用于解析变量关系。需牢记核心原则：**仅能证明相关关系，无法直接判定因果关系**。建模时不能只依赖 R^2 等指标，必须结合实际业务背景、行业逻辑综合判断模型的合理性与解释性。

总结：回归分析是机器学习的基石方法，其价值不仅在于预测未来，更在于通过数据量化变量间的关联，为业务决策提供可解释的依据。

第三次作业

整理分类算法的基本逻辑

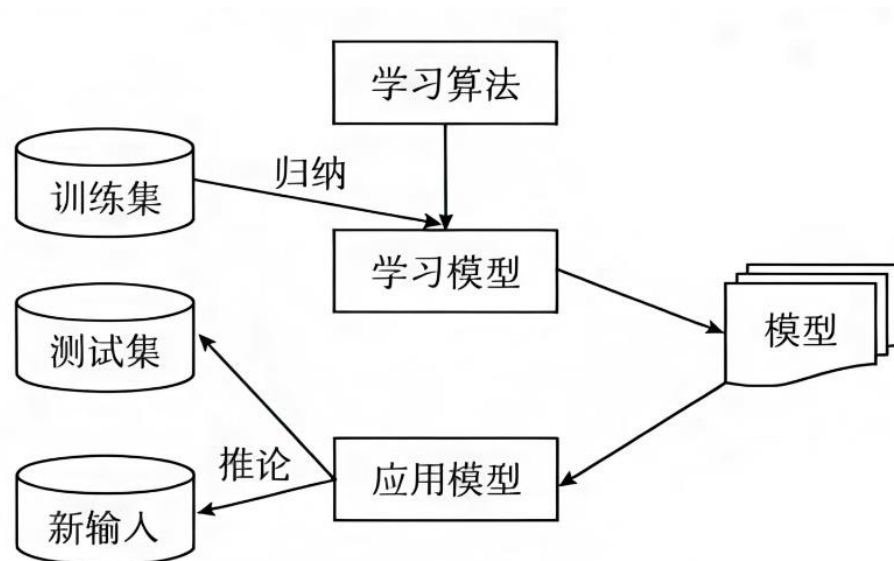


图 8-1 分类原理示意图

分类算法的基本逻辑就是**利用带标签的训练数据，学习一个能够推广到新样本的判别函数或概率模型**，从而在给定特征的情况下输出最可能的类别。

分类算法的几个基本概念

分类器：能够解释原始数据，能够预测新数据的模型；

训练集：用来训练模型的原始数据(划分出一部分)；

类标签：原始数据中已经给每个记录(对象)标记的类别；

测试集：用来测试模型的原始数据(划分出的一部分)；

开放集：使用训练好的模型(分类器)进行分类工作的新数据；

整理掌握KNN算法的思想与算法步骤

KNN算法的思想本质是局部近似

8.2 K-近邻

8.2.1 K-近邻原理

K-近邻 (K-Nearest Neighbor, KNN) 算法是一种基于实例的分类方法, 最初由 Cover 和 Hart 于 1968 年提出, 是一种非参数的分类技术。

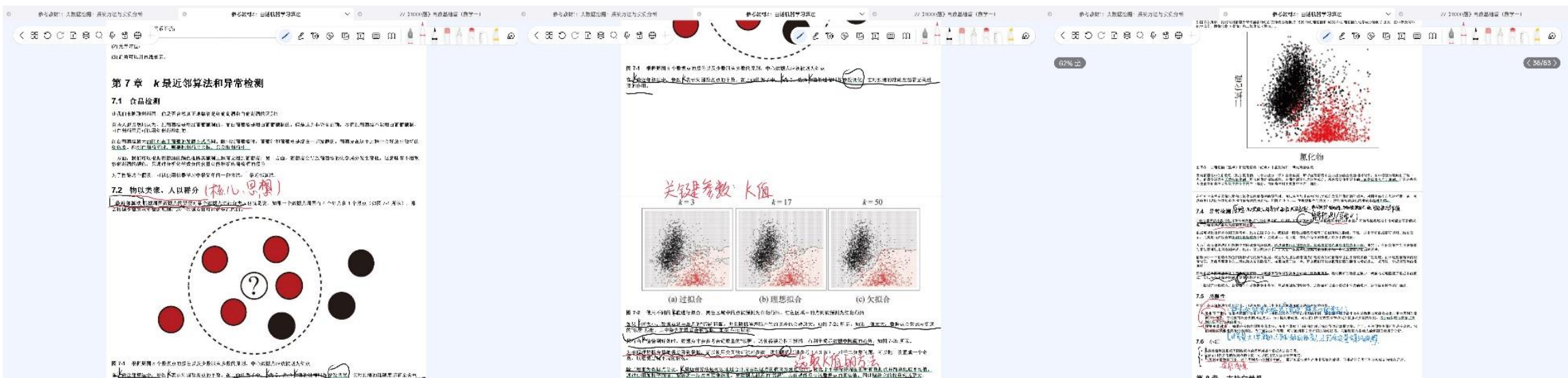
K-近邻分类方法通过计算每个训练样例到待分类样品的距离, 取和待分类样品距离最近的 K 个训练样例, K 个样品中哪个类别的训练样例占多数, 则待分类元组就属于哪个类别。使用最近邻确定类别的合理性用下面的谚语来说明: “如果走像鸭子, 叫像鸭子, 看起来像鸭子, 那就是鸭子。”

KNN算法基本步骤

KNN 算法具体步骤如下:

- 步骤 1: 初始化距离为最大值。
- 步骤 2: 计算未知样本和每个训练样本的距离 $dist$ 。
- 步骤 3: 得到目前 K 个最近邻样本中的最大距离 $maxdist$ 。
- 步骤 4: 如果 $dist$ 小于 $maxdist$, 则将该训练样本作为 K-最近邻样本。
- 步骤 5: 重复步骤 2~步骤 4, 直到未知样本和所有训练样本的距离都算完。
- 步骤 6: 统计 K 个最近邻样本中每个类别出现的次数。
- 步骤 7: 选择出现频率最大的类别作为未知样本的类别。

KNN算法笔记



读书笔记

决策树笔记

第9章 决策树

9.1 决策树幸存者

决策树是一种机器学习模型，它通过一系列决策规则来对数据进行分类。决策树的结构由根节点、内部节点和叶节点组成。每个节点包含一个特征和一个阈值，用于将数据分割成两个子集。叶节点包含一个类别标签。

决策树的优点是易于解释，并且对非线性数据具有良好的拟合能力。然而，决策树也容易过拟合，特别是在数据量较小的情况下。

```
graph TD
    Root[根节点] -- 是 --> Node1[是男性吗?]
    Node1 -- 是 --> Leaf1[生存概率为0]
    Node1 -- 否 --> Node2[月收入高于5000美元吗?]
    Node2 -- 是 --> Leaf2[生存概率为100%]
    Node2 -- 否 --> Leaf3[生存概率为75%]
```

9.3 生成决策树的过程

生成决策树的过程通常包括以下步骤：

- 选择特征和阈值，将数据集分割成两个子集。
- 递归地对每个子集进行分割，直到达到停止条件（如最大深度或最小样本数）。
- 对叶节点进行后处理，确定最终的类别标签。

第10章 随机森林

随机森林是一种集成学习方法，通过构建多个决策树并将它们的结果进行平均来提高模型的预测性能。随机森林的优点是能够处理高维数据，并且对过拟合具有较强的鲁棒性。

第四次作业

1.簇的概念

基本定义：簇是聚类分析中形成的组。聚类是将物理或抽象对象的集合分成由类似对象组成的多个类或簇的过程。

理解层次：

感官层面：“物以类聚，人以群分”。

逻辑层面：同一个簇中的对象相似度较高，与其他簇中的对象相似度较低。

量化层面：同一个分组中的记录越近越好，不同分组中的记录越远越好。

关键特性：簇的定义并非绝对精确，其最佳形态依赖于数据的特性和期望的结果。在空间分布上，簇可以呈现各种不同的形状。

2. 类度量的两种基本方法

衡量对象之间相似性或距离是聚类的核心，主要有两种方法：

欧式距离：

衡量的是两个点在空间中的绝对距离。

在二维平面上，点 $a(x_1, y_1)$ 与 $b(x_2, y_2)$ 间的欧氏距离公式为： $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ 。

关注的是数值上的差异。

余弦距离：

衡量的是两个向量在方向上的差异，通过计算它们夹角的余弦值来实现。

在二维空间中，向量 $A(x_1, y_1)$ 与 $B(x_2, y_2)$ 的夹角余弦公式为： $\cos\theta = (x_1 * x_2 + y_1 * y_2) / (\sqrt{x_1^2 + y_1^2} * \sqrt{x_2^2 + y_2^2})$ 。

关注的是方向的一致性，而非大小。当 $\cos\theta$ 越接近1，表示两个向量方向越一致，相似度越高。

3. K-means算法的原理和步骤

K-means是一种基于划分的聚类算法，其工作原理如下：

初始化：首先随机从数据集中选取 K 个点，作为初始的聚类中心。

分配：计算剩余各个样本到这 K 个聚类中心的距离，并将每个样本分配给距离最近的簇。

更新：重新计算每一簇内所有点的平均值，并将该平均值作为新的聚类中心。

迭代：重复执行 步骤2（分配）和 步骤3（更新），直到算法收敛。收敛的条件通常是相邻两次迭代中，点的归属不再改变或聚类中心的位置变化很小。

4. HC算法的过程

HC（层次聚类） 算法通过构建一个树状的层次结构来进行聚类。PPT中主要介绍了凝聚式（自底向上）的方法：

初始化： 将数据集 D 中的每个样本点都视为一个独立的簇。

循环合并：

在所有分属不同簇的点对中，找到距离最近的一对。

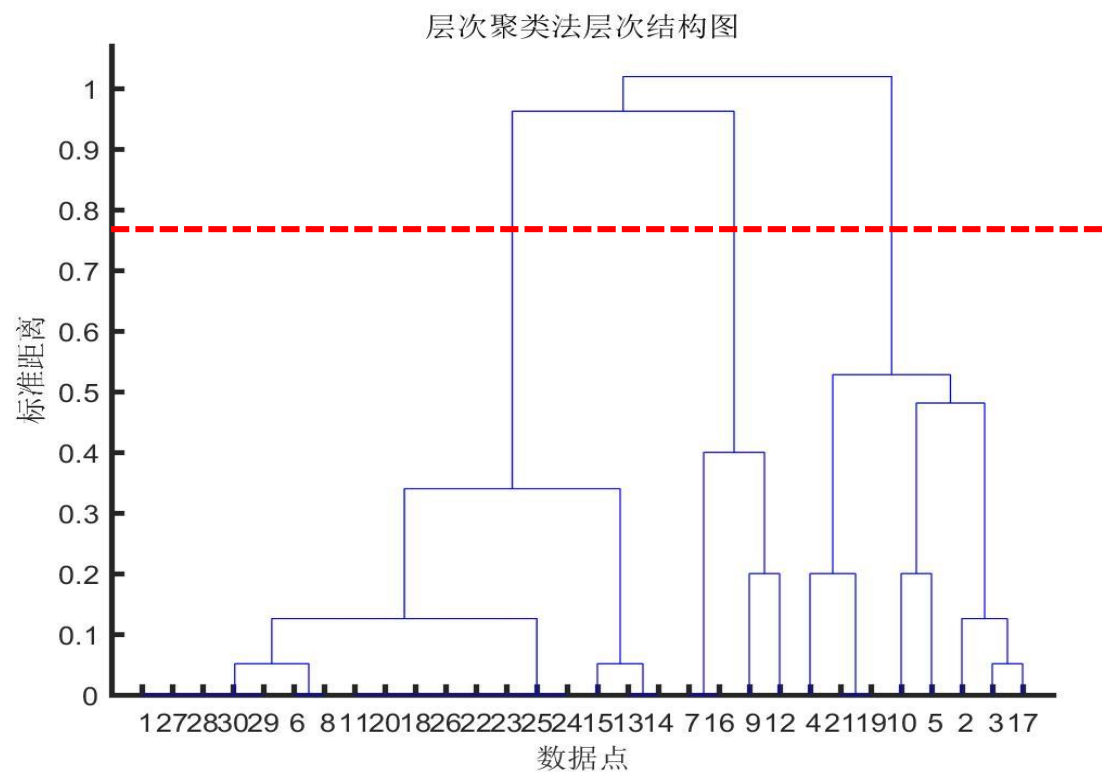
将这两个点所在的两个簇合并成一个新簇。

终止条件： 不断重复合并过程，直到簇的总数达到预设的目标数量 K 。

该算法最终会形成一个树状图，通过在这个图上画一条水平线，可以根据该线穿过的垂直线数量来确定最终的聚类数目。

5. 聚类结果图

图中红色水平虚线是划分类别的关键线。



K-means 算法读书笔记

1. 核心问题与解决思路

K-means 主要解决两个关键问题：

有多少个群组？

这是一个主观且需要权衡的问题。群组太少可能无法揭示有意义的模式；群组太多则可能导致群组间界限模糊，失去实际意义。

陡坡图 是确定最佳群组数量的常用工具。该图展示了“群组内散度”随群组数量增加而降低的趋势。曲线的拐弯处通常指示了最佳的群组数量，因为在此之后，增加群组带来的收益（散度降低）会急剧减少。

每个群组中有谁？

这个过程通过一个迭代优化的步骤来完成。

K-means 算法读书笔记

2. K-means 算法步骤

算法的核心是围绕“群组中心点”进行迭代，直到群组成员稳定。具体步骤如下：

- **初始化**：随机猜测 K 个群组的中心点位置，这些初始中心点被称为“伪中心点”。
- **分配**：计算每个数据点到各个伪中心点的距离，并将该数据点分配给距离最近的中心点所代表的群组。
- **更新**：根据当前分配到每个群组的所有数据点，重新计算该群组的中心点（即计算所有点的平均值），得到新的中心点。
- **迭代**：重复执行步骤2（分配）和步骤3（更新）。
- **收敛**：当在连续的迭代中，没有任何数据点改变其所属的群组时，算法停止，此时的群组划分即为最终结果。
- **关键点**：算法通过不断调整中心点和重新分配成员，使每个群组内部越来越紧凑（成员间距离小），群组之间越来越分离（群组间距离大）。

K-means 算法读书笔记

• 3. 算法局限性

书中明确指出了 K-means 算法的几个主要局限性：

硬聚类：每个数据点只能属于一个群组。如果一个点恰好位于两个群组的边界上，算法无法表达这种模糊性。

形状假设：算法假设群组是正圆形（或超球体）的。这是因为算法基于“到中心点的最短距离”进行分配，这自然倾向于形成圆形簇。对于非圆形（如椭圆形、月牙形）的簇，K-means 的效果会很差。

离散假设：算法假设群组是相互分离、不重叠、不嵌套的。它无法处理群组之间存在重叠或包含关系的情况。

第五次作业

1、关联选股的背景知识

行业关联选股法是把关联规则算法应用到股票市场分析中，用来发现行业或股票之间的联动关系。

股票市场中，不同行业之间往往存在产业链关系、政策影响关系、资金流关系和市场情绪关系。

例如，新能源汽车行业上涨时，锂电池、充电桩、稀有金属等行业可能会联动上涨；房地产行业上涨时，建材、家电、银行等行业也可能受到影响。

关联选股的基本思想是：把每个交易日看作一个事务，把行业或股票看作项，把当天上涨或满足条件的行业记为 1，不满足条件的记为 0。然后使用Apriori 算法找出经常共同出现的行业组合，并进一步分析行业之间的关联规则。

这种方法可以辅助发现行业联动关系，为行业轮动、板块分析和选股提供参考。但它只能说明统计关联，不能直接证明因果关系。

2、数据分析：矩阵行与列的含义

第五讲文件夹中的对应数据文件是一个典型的 0-1 矩阵示例。表中行是事务，列是项，元素是 0 或 1。

在行业关联选股中也可以使用同样的矩阵结构：

行表示交易日。每一行代表一天的市场情况，也就是一个事务。

列表示行业或股票。每一列代表一个行业或一只股票。

矩阵元素表示该行业或股票当天是否满足条件。

例如：

1 表示该行业当天上涨，或涨幅超过设定标准。

0 表示该行业当天没有上涨，或没有达到设定标准。

如果某一天新能源汽车、锂电池、充电桩三个行业都上涨，那么这一行中这三个行业对应的值为 1，其他未上涨行业对应的值为 0。

这种矩阵的意义是把现实中的交易数据转化为 Apriori 算法可以处理的事务数据。

3、运行程序

运行文件夹中的Python 程序。虽然它使用的是商品数据，但程序逻辑可以直接迁移到行业关联选股中。

程序主要步骤如下：

第一，导入相关库，包括pandas、TransactionEncoder、apriori、networkx 和 matplotlib。

第二，读取 products.csv 数据。

第三，把每一行中的“商品1、商品2、商品3”转化为一个事务列表。

第四，使用TransactionEncoder 把事务数据转化为 0-1 矩阵。

第五，调用Apriori 算法计算频繁项集。程序中设置的最小支持度是 0.02，表示某个组合至少出现 2 次。

第六，只保留 2 项集，并统计共现次数。

第七，使用networkx 构建商品关联网络图。

第八，把结果保存到 product_associations.csv。

如果应用到行业关联选股中，只需要把程序中的“商品”替换为“行业”：

products.csv 可替换为行业涨跌数据。

“商品1、商品2、商品3”可替换为某交易日上涨的行业列表。

每一行商品组合可理解为某一天共同上涨的行业组合。

Apriori 输出的频繁项集可理解为经常共同上涨的行业组合。

知识图谱中的节点表示行业，边表示两个行业之间存在共现关系，边的权重表示共同出现次数。

4、程序结果分析与思考

程序运行后会生成关联结果，记录了商品之间的共现关系。共现次数越高，说明两个商品在

同一事务中同时出现的次数越多，关联程度相对更明显。

例如结果中“无线鼠标和硬盘”共现 5 次，“电动牙刷和笔记本电脑”共现 5 次，“

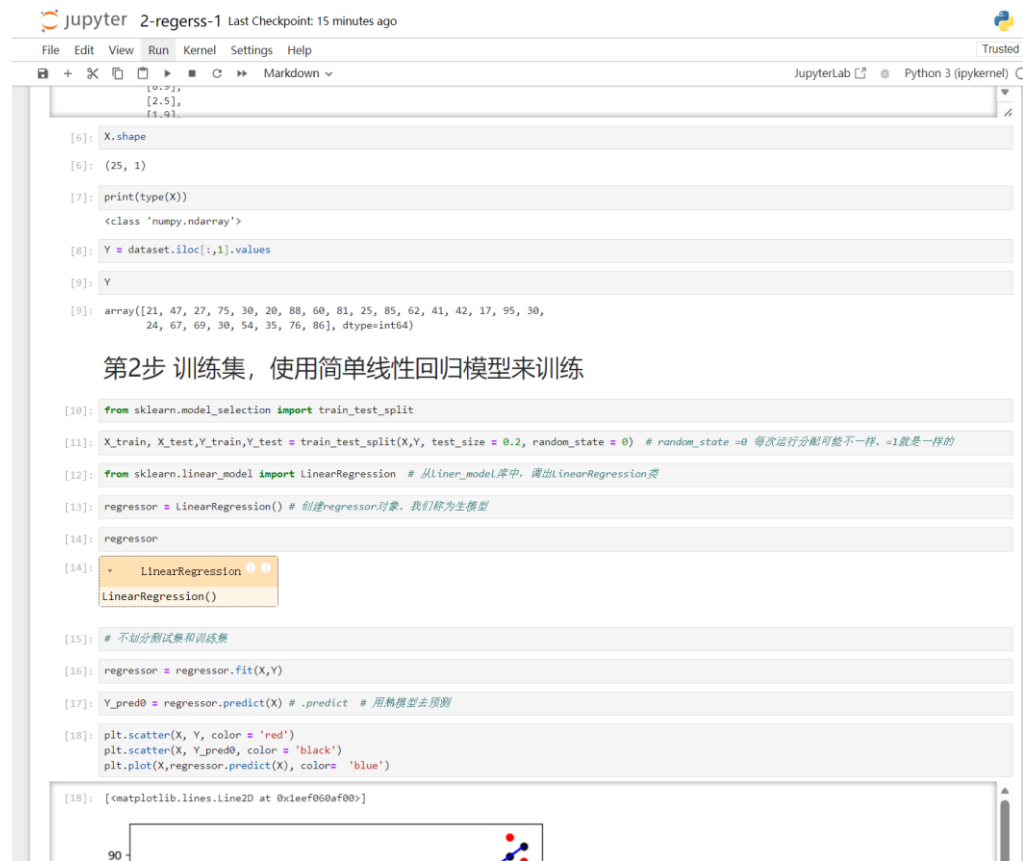
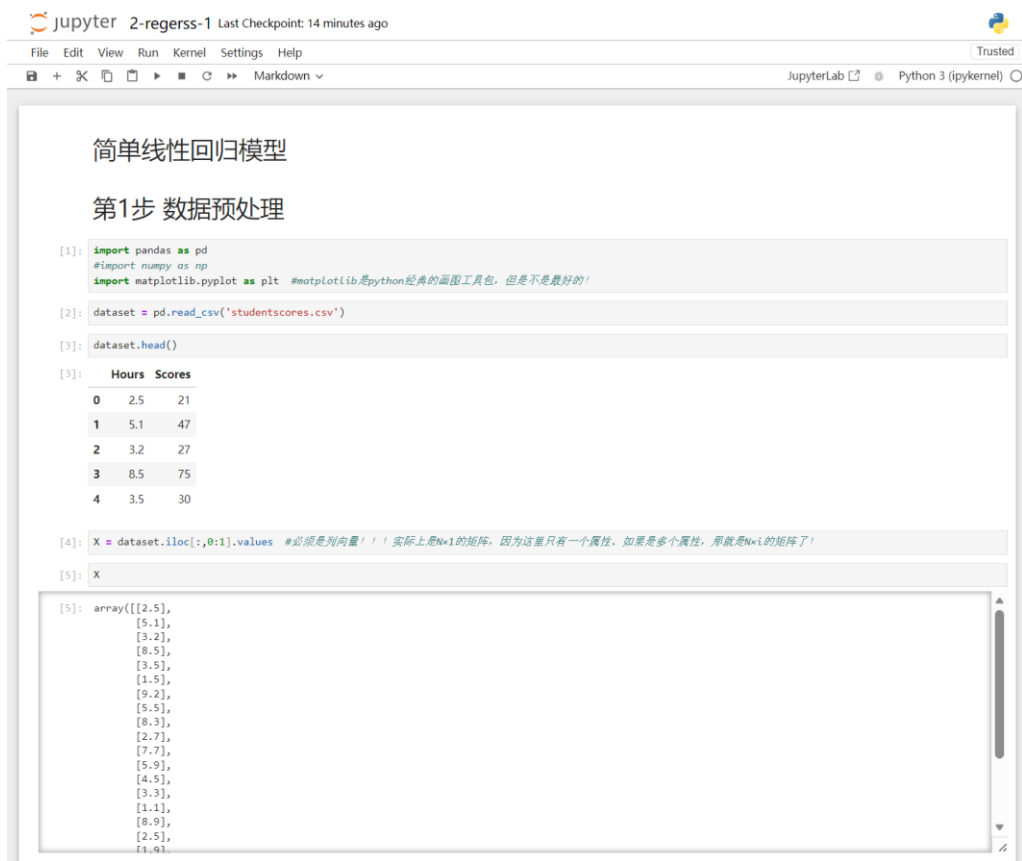
笔记本电脑和路由器”共现 4 次。这说明在当前样本数据中，这些商品组合出现得比较频繁，可以认为它们之间存在一定关联。

如果把这个过程应用到行业关联选股中，就可以把商品替换成行业，把一次购物记录替换成一个交易日。若某两个行业经常在同一天上涨，就说明它们可能存在联动关系。例如新能源汽车和锂电池经常共同上涨，可能是因为它们处在同一产业链中，容易受到相同政策、资金或市场情绪影响。

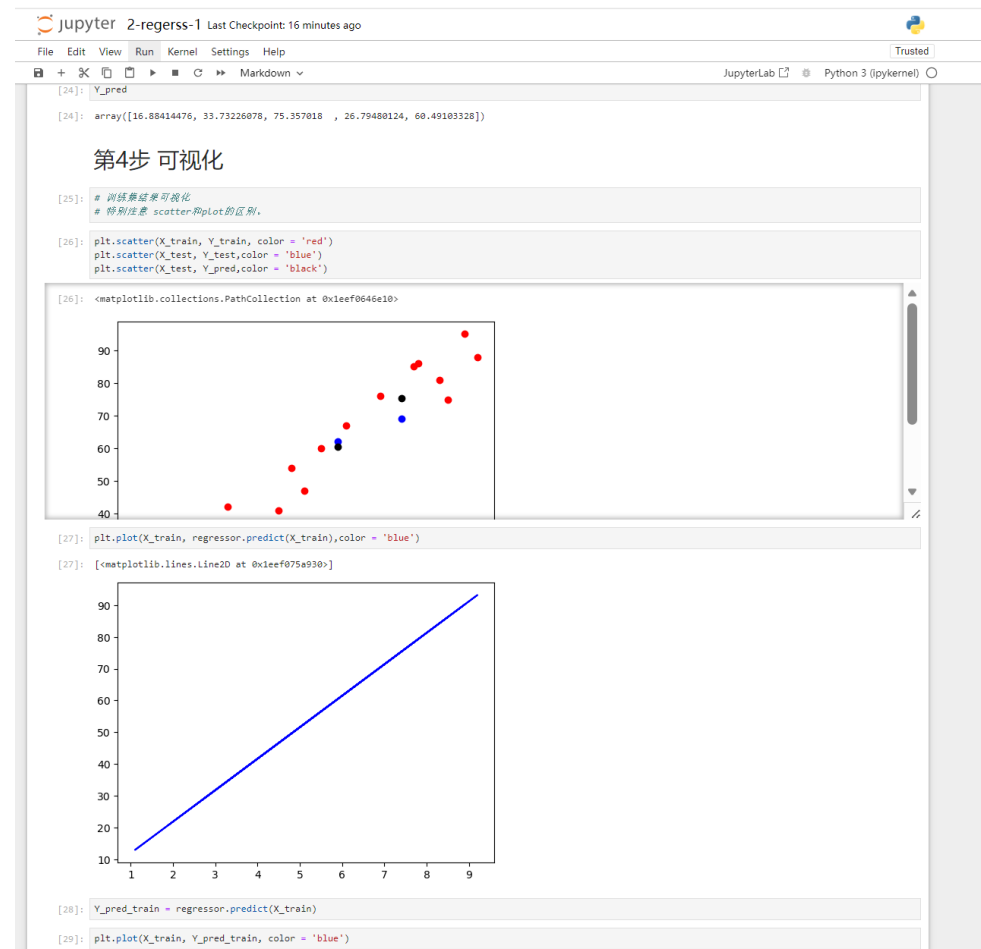
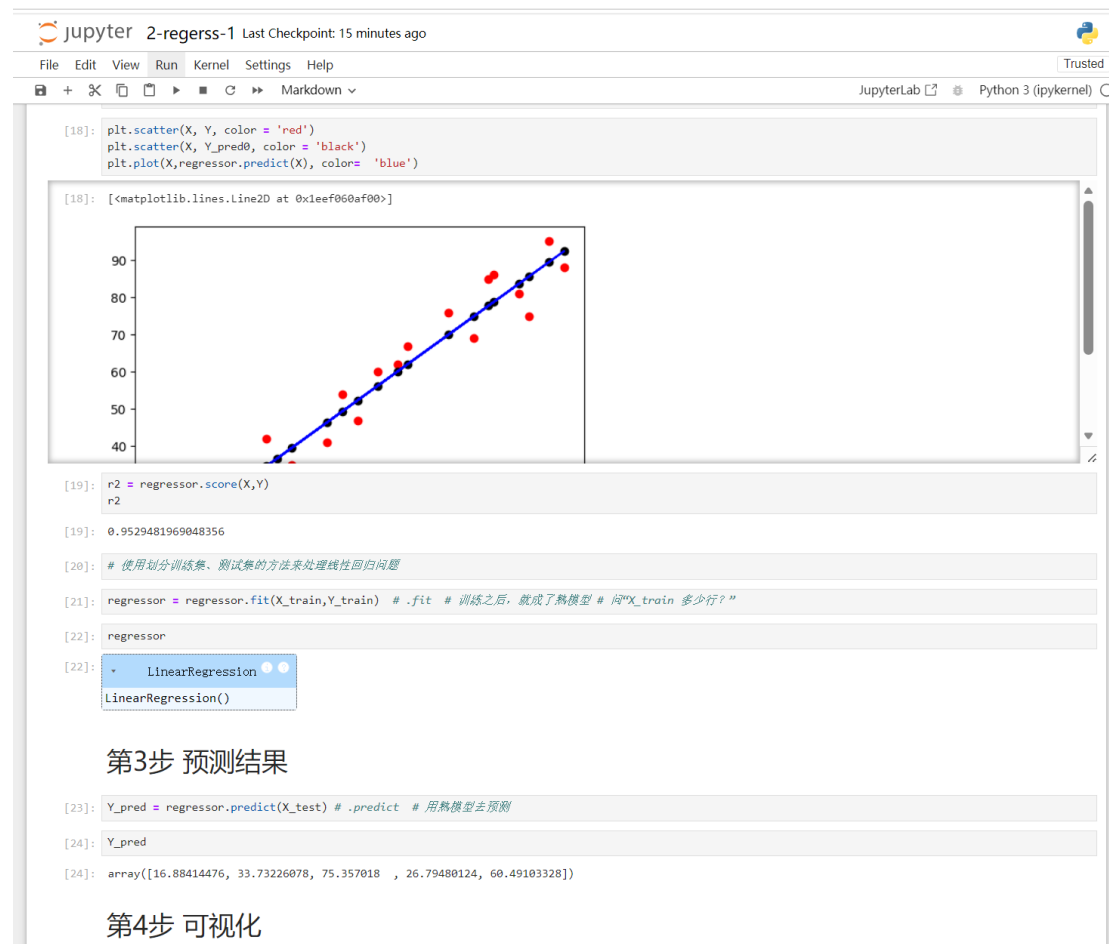
但需要注意，关联规则只能说明“经常一起出现”，不能直接说明因果关系。共现次数高不一定代表投资价值高，还要结合支持度、置信度、提升度以及行业背景进行判断。尤其是提升度，如果大于 1，说明这种关联比随机情况更强；如果接近 1，说明关联可能并不明显。因此，行业关联选股法可以作为辅助分析工具，用来发现行业之间的联动关系，但不能单独作为买卖股票的依据。

第六次作业

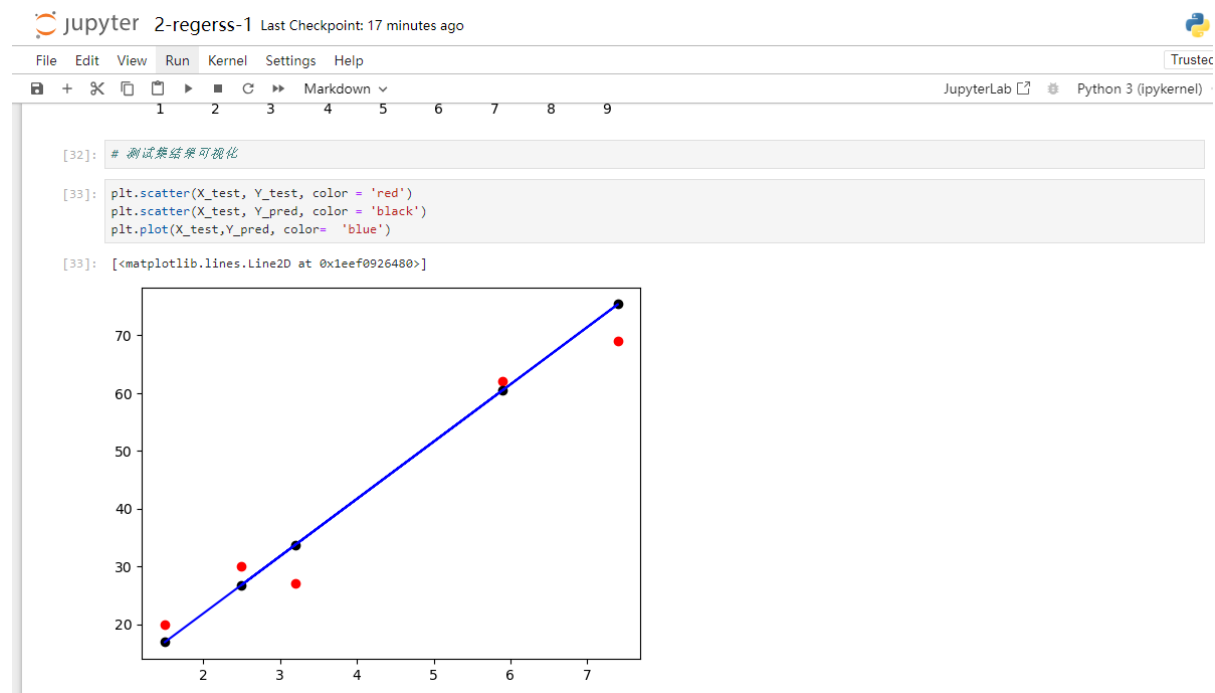
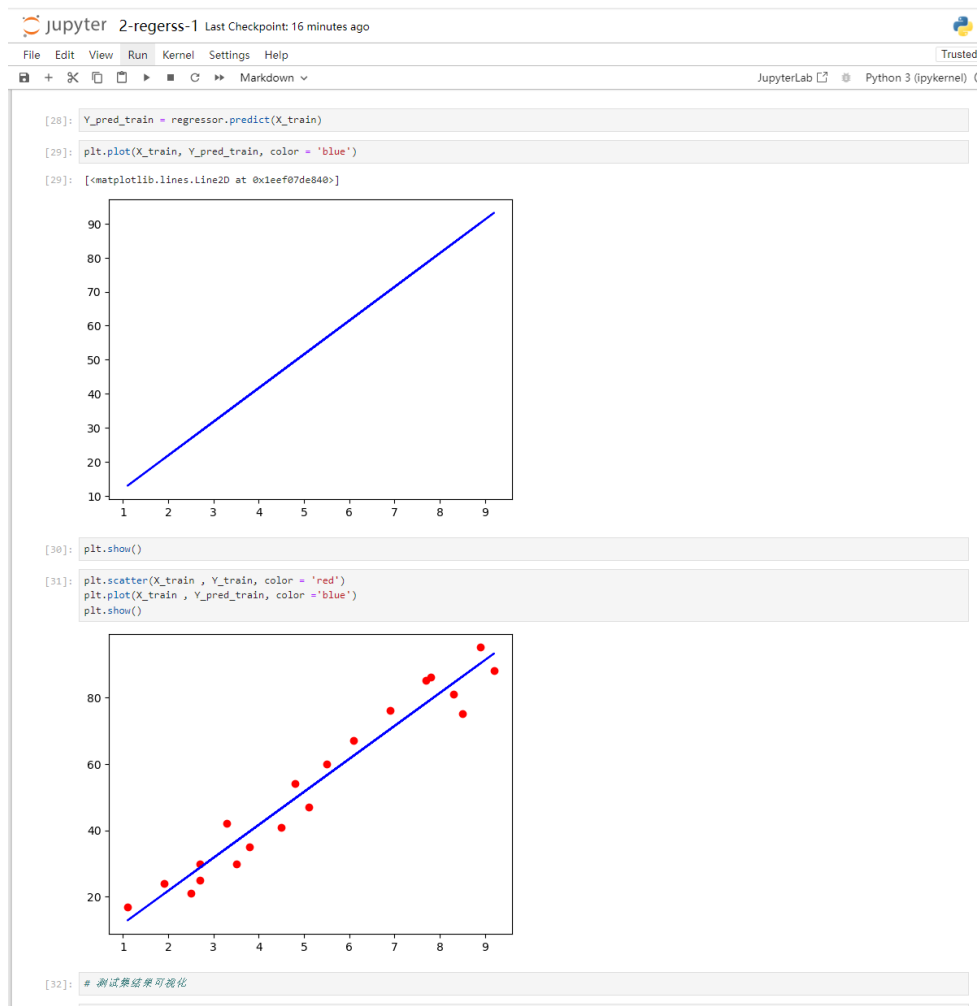
代码运行结果



代码运行结果



代码运行结果



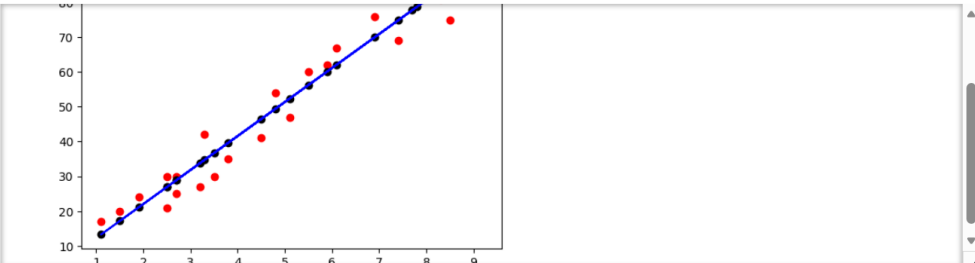
一、整理（数据处理的小细节）为什么X必须是列向量？变成行向量会怎么样呢？

X 实际上是一个矩阵，这个矩阵的大小是 25×1 ，25是行数，1是特征数，这样才可以运算！如果有M行，N个变量（属性/特征的话），那么这个矩阵就是： $M \times N$

二、如何显示生成的“熟模型”？ $y = b_0 + b_1 * x$, b_0 和 b_1 怎么获得？

第2步 训练集，使用简单线性回归模型来训练

```
[10]: from sklearn.model_selection import train_test_split
[11]: X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size = 0.2, random_state = 0) # random_state = 0 每次运行分配可能不一样，-1就是一样的
[12]: from sklearn.linear_model import LinearRegression # 从linear_model库中，调出LinearRegression类
[13]: regressor = LinearRegression() # 创建regressor对象，我们称为生模型
[14]: regressor
[14]: LinearRegression
LinearRegression()
[15]: # 不划分训练集和测试集
[16]: regressor = regressor.fit(X, Y)
[17]: Y_pred0 = regressor.predict(X) # .predict # 用熟模型去预测
[18]: plt.scatter(X, Y, color = 'red')
plt.scatter(X, Y_pred0, color = 'black')
plt.plot(X, regressor.predict(X), color = 'blue')
```



- 2 b_0 和 b_1 怎么生成

```
[34]: b1 = regressor.coef_ # 系数
b1
```

```
[34]: array([9.91065648])
```

```
[35]: b0 = regressor.intercept_ #截距 y = 2 + 9.9x
b0
```

```
[35]: 2.018160041434662
```

三、如何评价结果的好坏？（代码） R^2 的结果是什么？

在简单线性回归模型中，评价结果好坏的指标包括 R^2 （决定系数）、均方误差（MSE）、均方根误差（RMSE）、平均绝对误差（MAE）等。其中， R^2 是最常用的指标之一，它衡量了模型对数据变异的解释程度。

R^2 接近 1：说明自变量 x 能够很好地解释因变量 y 的变异，模型拟合效果好。

R^2 接近 0：说明自变量 x 几乎无法解释因变量 y 的变异，模型拟合效果差。

- 3 评价结果的好坏 R^2

```
[36]: R2_train = regressor.score(X_train,Y_train)
      R2_train
```

```
[36]: 0.9515510725211552
```

```
[37]: R2_test = regressor.score(X_test,Y_test)
      R2_test
```

```
[37]: 0.9454906892105354
```

四、调整训练/测试的比例，对结果 (R^2) 有什么影响？试试看

1. 训练集比例过小的影响

当训练集比例过小时，比如只占 20%，模型能学习到的数据规律有限。这就好比你看只看了少量的例题就去参加考试，很难掌握全部知识点。此时，模型在训练集上的 R^2 可能看起来还不错，但在测试集上的 R^2 会比较低，因为模型没有充分学习到数据中的真实关系，出现过拟合的可能性较小，但容易出现欠拟合。

2. 训练集比例过大的影响

要是训练集比例过大，例如达到 90%，模型在训练集上会学习得非常充分，甚至可能会把数据中的噪声也当作规律来学习。这就像你把练习题的答案都背下来了，但遇到新的考试题（测试集）就不会做了。这种情况下，训练集上的 R^2 会很高，接近 1，但测试集上的 R^2 可能会明显下降，出现过拟合现象。

3. 合适的训练/测试比例

一般来说，常见的训练/测试比例是 70%/30%、80%/20% 或者 75%/25%。这些比例能让模型充分学习数据规律和避免过拟合之间取得较好的平衡。不过，具体选择哪种比例，还需要根据数据集的大小和特点来决定。如果数据集很小，可能需要采用交叉验证的方法来更准确地评估模型性能。

第七次作业

多元线性回归

王荣崇

信管2301

202321054020

导入数据集

```
[3]: dataset = pd.read_csv('50_Startups.csv')  
dataset.head()
```

```
[3]:
```

	R&D Spend-X1	Administration-X2	Marketing Spend-X3	State-X4	Profit-Y
0	165349.20	136897.80	471784.10	New York	192261.83
1	162597.70	151377.59	443898.53	California	191792.06
2	153441.51	101145.55	407934.54	Florida	191050.39
3	144372.41	118671.85	383199.62	New York	182901.99
4	142107.34	91391.77	366168.42	Florida	166187.94

属性数量

这个 50_Startups 数据集，一共 5 个字段
(属性)：

R&D Spend

Administration

Marketing Spend

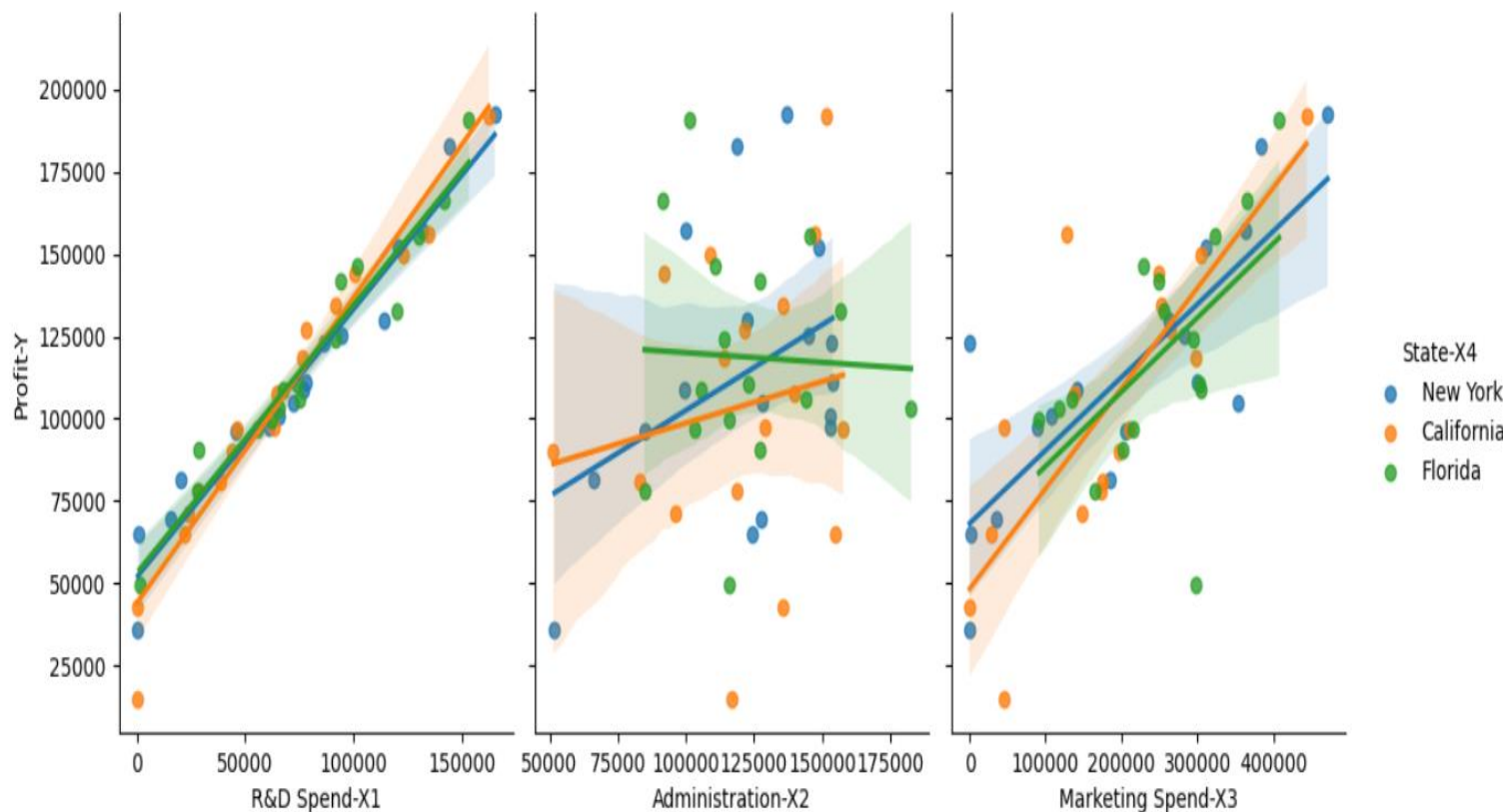
State

Profit

自变量共 4 个，因变量 1 个，总计 5 个属性。

分类变量：State

可视化分类变量(城市)和因变量(营收)之间的关系。



不同城市对企业营收有明显影响，说明分类变量State是重要影响因素。New York 和 California 的企业整体营收更高，Florida 相对略低。同一城市内部企业营收差异也较大，说明地理位置不是唯一影响因素。

分类变量转化为onehot过程

1.识别分类特征：本次数据里，State（城市/州）是无序分类变量（New York、California、Florida，类别之间没有大小、高低的顺序关系），无法直接放入线性回归模型计算，必须数字化处理。

2.标签编码（过渡步骤）：先用LabelEncoder，把文本类别的城市名称，先批量转换为整数数字：0、1、2，仅完成文字到数字的格式转换，此时绝对不能直接建模，否则模型会错误认为 $2 > 1 > 0$ ，引入虚假大小关系。

3.独热（OneHot）编码转换使用ColumnTransformer精准指定仅对分类列做编码：

为 3 个城市分别新建独立的 0/1 特征列

某样本属于该城市则标记为1，不属于则标记为0

其余数值型特征（研发投入、管理成本、营销费用）完整保留不变

4.消除虚拟变量（多重共线性）：陷阱N 个分类会生成 N 个独热列，回归时会出现完全多重共线性，因此手动删除 1 个基准类别列（代码里 $X = X[:,3:]$ ，舍弃第一列基准哑变量），避免模型矩阵奇异、回归系数失真。

5.得到最终可用数据集：最终所有特征全部为纯数值、无虚假顺序、无共线性问题，可直接用于多元线性回归训练。

评估结果R²

```
R2 = regressor.score(X,Y)
```

```
R2
```

```
0.9490884301964866
```

模型 $R^2 = 0.949$ ，说明模型能解释 94.9% 的利润变化，拟合效果极佳。

如何获得系数(属性的权重), 以及截距?怎么分析这个结果?最终的数学公式应该是什么?

```
bi = regressor.coef_ # 系数(权重)  
bi
```

```
array([0.77884104, 0.0293919 , 0.03471025])
```

返回权重

```
b0 = regressor.intercept_  
b0
```

```
42989.00816508672
```

返回截距

最终数学公式:

$$y=42989.01+0.7788a+0.0294b +0.0347c$$

y: 预测企业利润

a: 研发投入

b: 行政管理费用

c: 营销推广费用

结果分析: 模型截距为 42989.01, 代表所有投入为 0 时企业的基准利润水平; 三个自变量中, 研发投入系数高达 0.7788, 呈极强正相关, 是提升利润最核心、贡献最大的驱动因素; 营销投入系数 0.0347、管理成本系数 0.0294, 二者仅存在微弱的正向促进作用, 对利润的拉动效果十分有限, 整体来看企业利润高度依赖研发投入, 管理与营销投入的盈利转化效率偏低。

分类变量的存在，对多元线性回归带来的影响

城市（State）这类无序分类变量，若不做正确处理，将无法直接参与多元线性回归；如果错误直接标签化赋值，还会人为引入虚假的大小顺序，误导模型计算、让回归系数和结果严重失真。而经过One-Hot 独热编码并规避虚拟变量陷阱后，分类变量可以有效量化不同城市的基础利润差异，补充地域维度的影响，提升模型整体拟合效果与解释能力；但如果编码处理不当，又容易带来多重共线性问题，干扰模型稳定。因此，分类变量的正确预处理，是保障多元线性回归结果准确可靠的关键前提。

能否把分类变量直接lable化，也就是变成0、1、2后直接计算？不转化称one-hot合理吗？

- 不合理，绝对不能直接 Label 编码（0、1、2）后直接建模。
- 本次案例里的城市（State）是无序分类变量：New York、California、Florida 三者之间没有高低、大小、先后的等级顺序，只是不同类别标签。
- 如果直接映射为 0、1、2，线性回归模型会默认认为「 $2 > 1 > 0$ 」，错误判定类别之间存在数值递增关系，人为制造出根本不存在的排序差异，导致回归系数、权重、预测结果全部严重偏差、结论完全失去参考意义。
- 只有有序分类变量（如：低 / 中 / 高、等级 1/2/3），本身存在天然顺序，才可以考虑 Label 编码。
- 对于城市这类无序分类，必须做 One-Hot 独热编码，才能公平区分不同类别、避免虚假排序，保证多元线性回归的科学性与准确性。

第八次作业

加入性别因素

未加入性别因素

```
from sklearn.metrics import confusion_matrix # 这个工具也是现成的
```

```
cm = confusion_matrix(y_test, y_pred) # y_test就是测试集原来的标签（人给的）；y_pred就是结果
```

- 就长下面这个样子
- 简单来说，对角线上的，就是全部判断对的，所有4个值加起来，就是测试数据总个数100
- 64对应的是 (0, 0)，0全部判断为0；4代表，有4个应该是0判断成1；3代表，有3个应该是1判断成0
- 这样就是100个当中，7个判断错了，93个判断对了

```
cm
```

```
array([[55,  3],  
       [ 1, 21]], dtype=int64)
```

- 关于混淆矩阵，更加详细的内容，我特别建议大家参考下面的博客，这样可以了解的清楚一点
- 博客地址：<https://www.jianshu.com/p/27228ad417d4>

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # 用对角线上的和，除以总
```

```
accuracy
```

```
0.95
```

加入性别因素

```
from sklearn.metrics import confusion_matrix
```

```
cm = confusion_matrix(y_test,y_pred)  
cm
```

```
array([[55,  3],  
       [ 1, 21]], dtype=int64)
```

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])  
accuracy
```

```
0.95
```

准确率均为0.95，加入性别因素无明显变化

比较KNN与Logistic在这个数据案例上的结果差异，哪个算法对这个数据更有效

Logistic回归结果

第4步：评估预测

我们预测了测试集。现在我们将评估逻辑回归模型是否正确的学习和理解。因此这个混淆矩阵将包含我们模型的正确和错误的预测。

生成混淆矩阵

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
```

```
[y_test, y_pred]
```

```
[array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
        0, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
        1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
        0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 1], dtype=int64),
 array([0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1,
        0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0,
        1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1,
        0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1], dtype=int64)]
```

```
cm
```

```
array([[57,  1],
       [ 5, 17]], dtype=int64)
```

```
p = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[1,1]+cm[0,1]+cm[1,0])
p
```

```
0.925
```

KNN结果为0.95与Logistic结果为0.925。KNN在这个数据案例上效果更好

训练集的比例，如何影响结果

test_size = 0.1

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) #
```

```
accuracy
```

```
0.925
```

test_size = 0.2

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) #
```

```
accuracy
```

```
0.95
```

test_size = 0.3

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) #
```

```
accuracy
```

```
0.9166666666666666
```

test_size = 0.4

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) #
```

```
accuracy
```

```
0.90625
```

训练集比例会影响模型效果，训练集越充分效果越稳定，test_size = 0.2时模型效果最好

超参数K如何影响结果，设置K=1、3、7、9

K = 1

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1]) # ./
```

```
accuracy
```

```
0.8875
```

K = 3

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
```

```
accuracy
```

```
0.95
```

K = 5

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
```

```
accuracy
```

```
0.95
```

K = 7

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
```

```
accuracy
```

```
0.95
```

K = 9

```
accuracy = (cm[0,0]+cm[1,1])/(cm[0,0]+cm[0,1]+cm[1,0]+cm[1,1])
```

```
accuracy
```

```
0.95
```

1.K=1时，准确率为0.8875

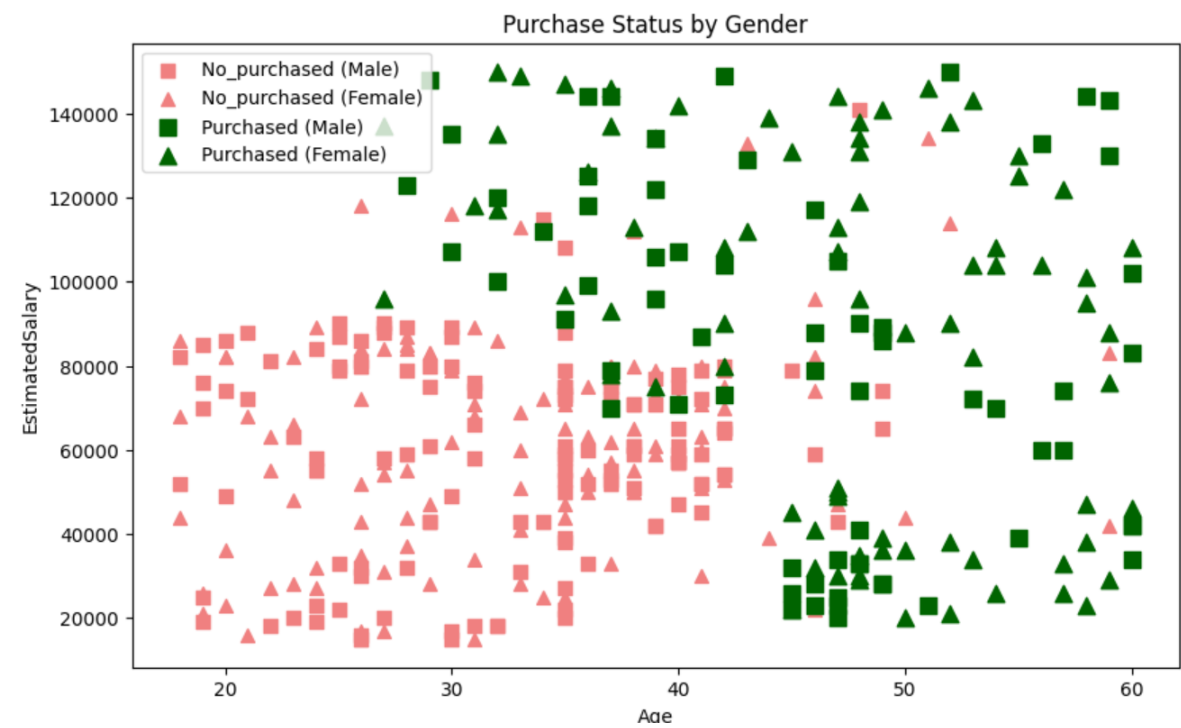
过拟合：容易受到噪声点和异常值的影响

决策边界非常不规则，锯齿状明显
泛化能力差，测试集准确率低

2.K=3,5,7,9时，准确率为0.95

3.合适的K值可以提高模型准确性

对可视化图进行修改，把性别变量考虑到可视化中



组合	颜色	形状	含义
未购买+男	浅红/红	方形 ■	没买产品的男性
未购买+女	浅红/红	三角 ▲	没买产品的女性
购买+男	深绿/绿	方形 ■	买了产品的男性
购买+女	深绿/绿	三角 ▲	买了产品的女性

通过颜色区分是否购买
通过点的形状区分性别

第九次作业

1.加入性别因素后acc的变化

```
# =====  
# 作业 1: 加入性别因素  
# =====  
print("=" * 50)  
print("作业 1: 加入性别因素")  
  
# 对性别进行数值编码  
le = LabelEncoder()  
gender_encoded = le.fit_transform(dataset['Gender']) # Female=0, Male=1  
  
# 构建包含性别的特征矩阵 (Gender, Age, EstimatedSalary)  
X_gender = np.column_stack((gender_encoded, dataset.iloc[:, [2, 3]].values))  
  
X_train_g, X_test_g, y_train_g, y_test_g = train_test_split(  
    X_gender, y, test_size=0.2, random_state=RANDOM_STATE)  
  
sc_g = StandardScaler()  
X_train_g_scaled = sc_g.fit_transform(X_train_g)  
X_test_g_scaled = sc_g.transform(X_test_g)  
  
dt_gender = DecisionTreeClassifier(criterion='entropy', random_state=RANDOM_STATE)  
dt_gender.fit(X_train_g_scaled, y_train_g)  
y_pred_g = dt_gender.predict(X_test_g_scaled)  
acc_gender = accuracy_score(y_test_g, y_pred_g)  
cm_gender = confusion_matrix(y_test_g, y_pred_g)  
  
print(f"加入性别后的准确率: {acc_gender:.4f}")  
print("混淆矩阵:")  
print(cm_gender)
```

```
=====  
作业 1: 加入性别因素  
加入性别后的准确率: 0.9125  
混淆矩阵:  
[[54  4]  
 [ 3 19]]
```

特征组合	决策树准确率
仅Age + EstimatedSalary	0.9250
Age + EstimatedSalary + Gender	0.9125

Acc变化准确率下降可能原因如下:
特征与目标弱相关甚至无关
决策树容易过拟合

2. 不进行特征缩放，预测的准确性de1变化

```
# 作业 2: 不进行特征缩放 (决策树)
# =====
print("\n" + "=" * 50)
print("作业 2: 不进行特征缩放 (决策树)")

# 为公平对比, 重新划分一次 (无缩放)
X_train_raw, X_test_raw, y_train_raw, y_test_raw = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE)

dt_no_scale = DecisionTreeClassifier(criterion='entropy', random_state=RANDOM_STATE)
dt_no_scale.fit(X_train_raw, y_train_raw)
y_pred_ns = dt_no_scale.predict(X_test_raw)
acc_no_scale = accuracy_score(y_test_raw, y_pred_ns)
cm_no_scale = confusion_matrix(y_test_raw, y_pred_ns)

print(f"不缩放的准确率: {acc_no_scale:.4f}")
print("混淆矩阵:")
print(cm_no_scale)

# 缩放后的决策树 (用于对比)
dt_scaled = DecisionTreeClassifier(criterion='entropy', random_state=RANDOM_STATE)
dt_scaled.fit(X_train_scaled, y_train)
y_pred_dt_scaled = dt_scaled.predict(X_test_scaled)
acc_dt_scaled = accuracy_score(y_test, y_pred_dt_scaled)

print(f"缩放的决策树准确率: {acc_dt_scaled:.4f}")
print("结论: 对决策树而言, 特征缩放理论上不影响结果 (若固定随机种子)。")
```

```
=====
作业 2: 不进行特征缩放 (决策树)
不缩放的准确率: 0.9000
混淆矩阵:
[[ 53  5]
 [ 3 19]]
缩放的决策树准确率: 0.9000
结论: 对决策树而言, 特征缩放理论上不影响结果 (若固定随机种子)。
```

不缩放准确率	0.9000
缩放准确率	0.9000

决策树只关心特征值的顺序, 不关心绝对值大小。
它的分裂规则形如Age <= 0.5 (标准化后) 或Age <= 30 (原始值),
但这两个条件找到的样本分割完全等价。只要你在两次训练时:

数据划分完全相同 (相同random_state)

模型随机种子相同 (random_state=RANDOM_STATE)

那么训练出的树结构必然一致, 预测结果当然一样。

3 比较KNN、Logistic、决策树，随机森林在该数据集上的表现

代码运行：

```
# 作业 3: 比较四种模型
# =====
print("\n" + "=" * 50)
print("作业 3: 四种模型表现对比")

models = {
    'KNN (k=5)': KNeighborsClassifier(n_neighbors=5),
    'Logistic Regression': LogisticRegression(random_state=RANDOM_STATE),
    'Decision Tree': DecisionTreeClassifier(criterion='entropy', random_state=RANDOM_STATE),
    'Random Forest': RandomForestClassifier(criterion='entropy', n_estimators=100, random_state=RANDOM_STAT)
}

results = {}
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)
    results[name] = {'accuracy': acc, 'confusion_matrix': cm}
    print(f"{name} 准确率: {acc:.4f}")
    print(f"混淆矩阵:\n{cm}\n")

# 绘制对比柱状图
names = list(results.keys())
accs = [results[n]['accuracy'] for n in names]

plt.figure(figsize=(8, 5))
bars = plt.bar(names, accs, color=['skyblue', 'lightgreen', 'salmon', 'gold'])
plt.ylim(0.8, 1.0)
plt.ylabel('Accuracy')
plt.title('Model Comparison on Social Network Ads')
for bar, acc in zip(bars, accs):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005, f'{acc:.4f}', ha='center')
plt.tight_layout()
plt.show()
```

3. 比较KNN、Logistic、决策树，随机森林在该数据集上的表现。

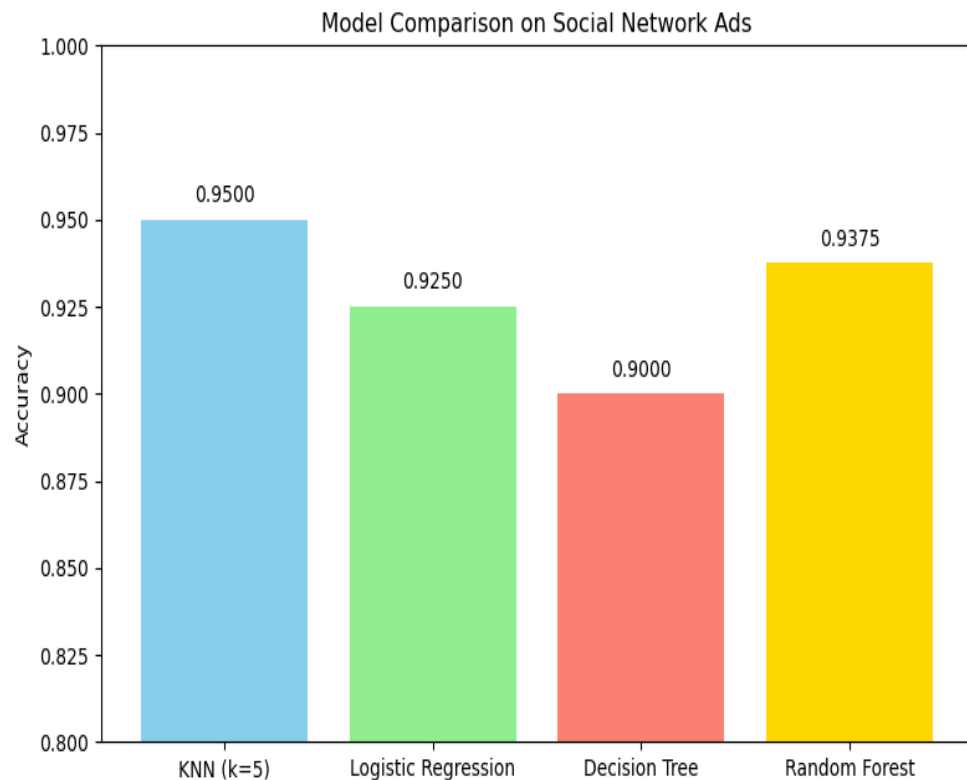
运行结果：

```
KNN (k=5) 准确率: 0.9500  
混淆矩阵:  
[[55  3]  
 [ 1 21]]
```

```
Decision Tree 准确率: 0.9000  
混淆矩阵:  
[[53  5]  
 [ 3 19]]
```

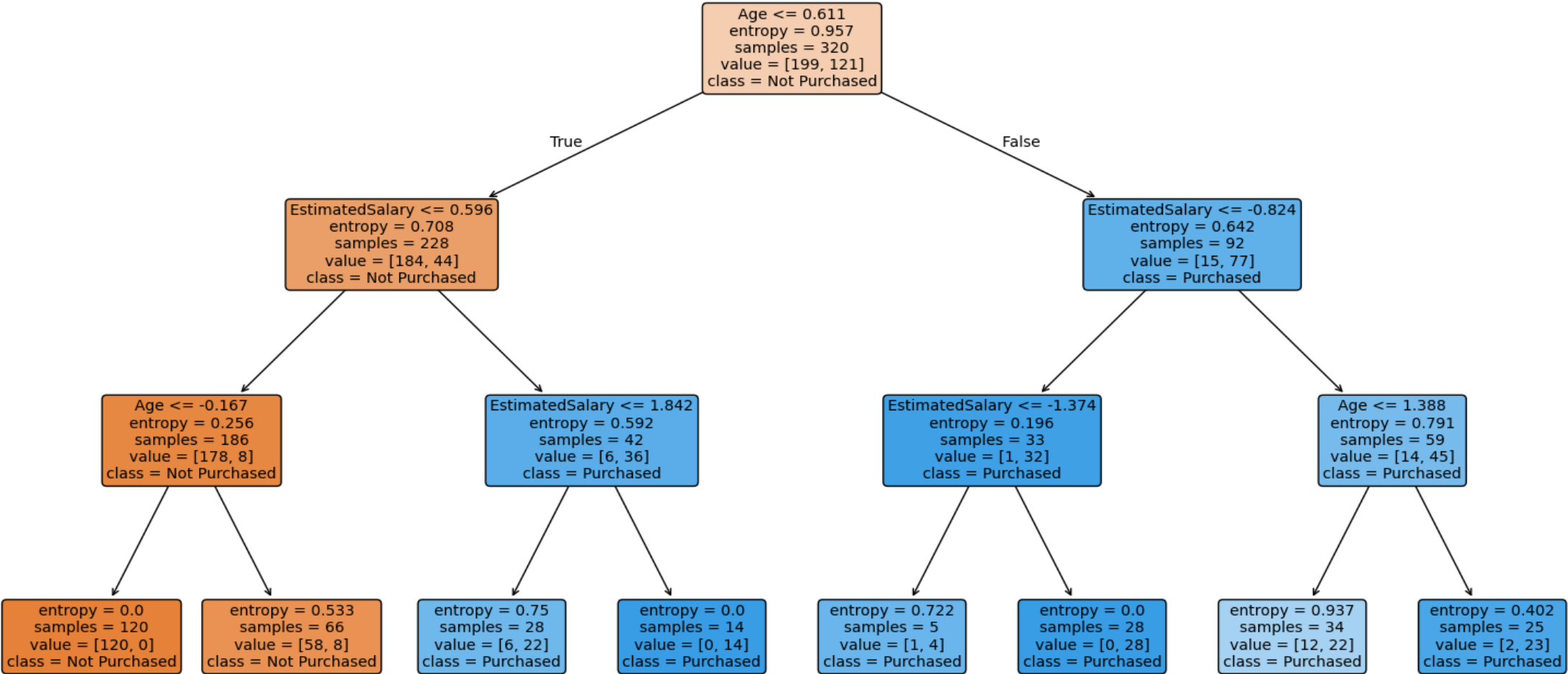
```
Logistic Regression 准确率: 0.9250  
混淆矩阵:  
[[57  1]  
 [ 5 17]]
```

```
Random Forest 准确率: 0.9375  
混淆矩阵:  
[[55  3]  
 [ 2 20]]
```



4.决策树

Decision Tree (max_depth=3)



第十次作业

一、用花萼的数据聚类

```
[39]: import matplotlib.pyplot as plt
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import load_iris

# 加载数据
iris = load_iris()
X = iris.data # 所有4列数据
X_sepal = iris.data[:, 0:2] # 花萼数据: 第0列(花萼长度) 和第1列(花萼宽度)
X_petal = iris.data[:, 2:4] # 花瓣数据: 第2列和第3列

print("数据形状:", X.shape)
print("花萼数据形状:", X_sepal.shape)

数据形状: (150, 4)
花萼数据形状: (150, 2)

[40]: # 使用花萼数据 (sepal length, sepal width)
estimator_sepal = KMeans(n_clusters=3, random_state=42)
estimator_sepal.fit(X_sepal)

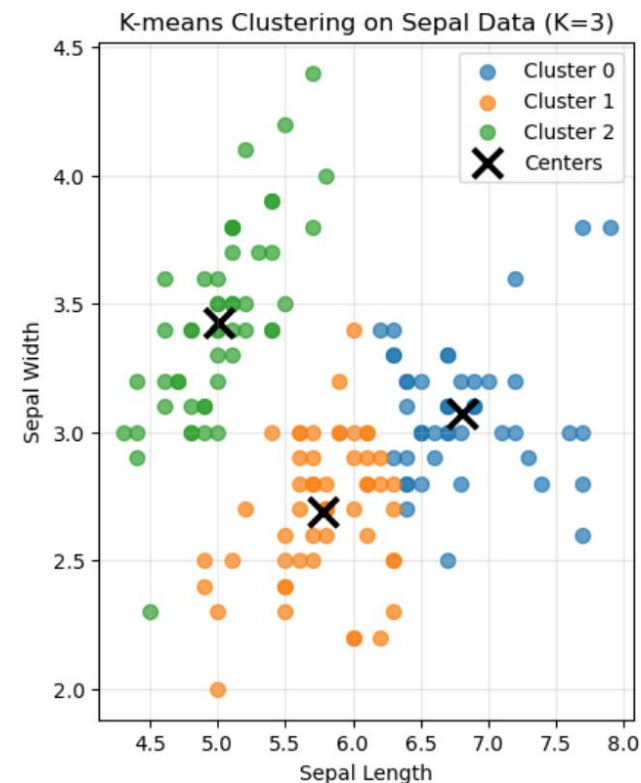
# 获取聚类标签和中心
labels_sepal = estimator_sepal.labels_
centers_sepal = estimator_sepal.cluster_centers_

print("\n=== 花萼数据聚类结果 ===")
print("聚类中心:\n", centers_sepal)
print("前10个样本的聚类标签:", labels_sepal[:10])

# 可视化
plt.figure(figsize=(16, 6))

plt.subplot(1, 3, 1)
# 分别绘制三个簇
for i in range(3):
    mask = labels_sepal == i
    plt.scatter(X_sepal[mask, 0], X_sepal[mask, 1],
               label=f'Cluster {i}', alpha=0.7, s=50)

# 绘制聚类中心
plt.scatter(centers_sepal[:, 0], centers_sepal[:, 1],
           c='black', marker='x', s=200, linewidth=3, label='Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Clustering on Sepal Data (K=3)')
plt.legend()
plt.grid(True, alpha=0.3)
```



仅凭花萼特征难以完美区分所有样本，若加入花瓣特征可显著提升聚类效果。

二、用所有数据聚类

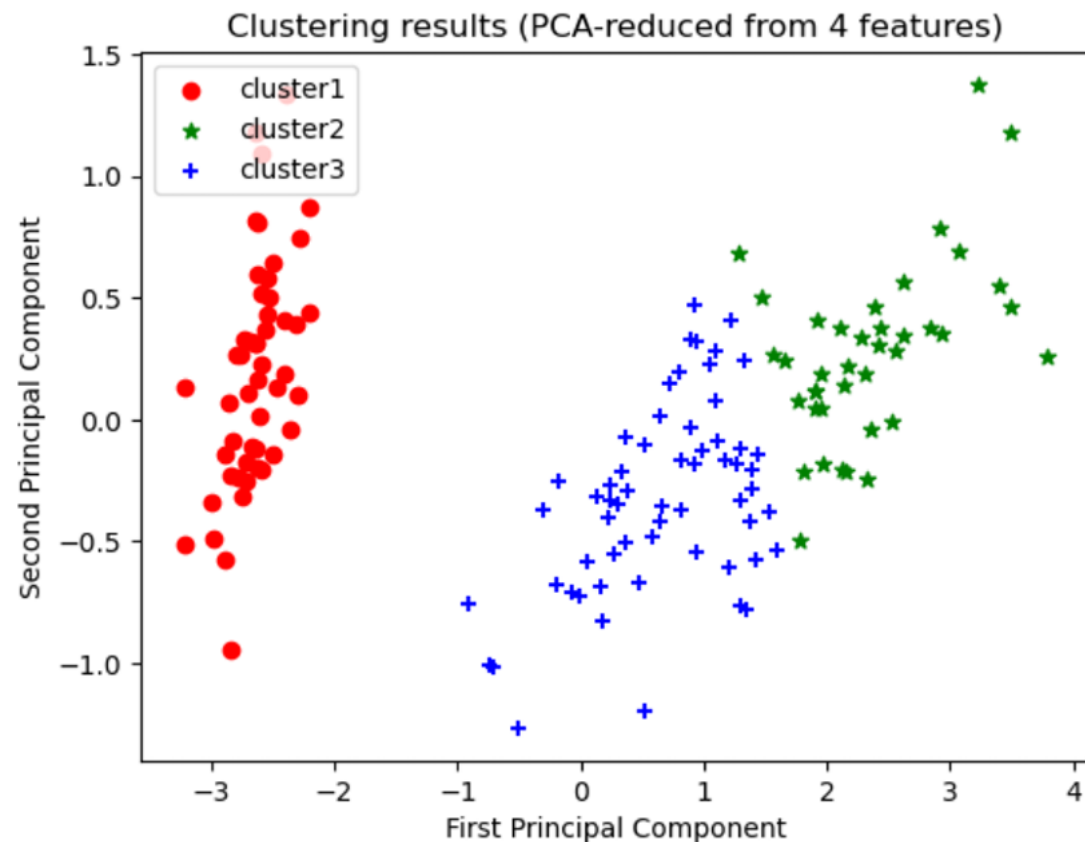
```
#使用PCA降维到2D进行可视化
from sklearn.decomposition import PCA

X_3 = iris.data[:, :] # 使用全部4个特征
estimator.fit(X_3)
label_pred = estimator.labels_

# PCA降维到2维
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_3)

x0 = X_pca[label_pred == 0]
x1 = X_pca[label_pred == 1]
x2 = X_pca[label_pred == 2]

plt.scatter(x0[:, 0], x0[:, 1], c='red', marker='o', label='cluster1')
plt.scatter(x1[:, 0], x1[:, 1], c='green', marker='*', label='cluster2')
plt.scatter(x2[:, 0], x2[:, 1], c='blue', marker='+', label='cluster3')
plt.xlabel('First Principal Component')
plt.ylabel('Second Principal Component')
plt.title('Clustering results (PCA-reduced from 4 features)')
plt.legend(loc=2)
plt.show()
```



红色簇（Setosa）完全独立，分离度极高；另外两簇虽有部分重叠，但整体界限清晰。这说明 $K=3$ 的选择合理，算法成功捕捉到了数据在高维空间中的自然分组特征。

三、这里的K=3，把K改成2

```
[42]: # 使用花瓣数据，但将K改为2
estimator_sepal_k2 = KMeans(n_clusters=2, random_state=42)
estimator_sepal_k2.fit(X_sepal)

labels_sepal_k2 = estimator_sepal_k2.labels_
centers_sepal_k2 = estimator_sepal_k2.cluster_centers_

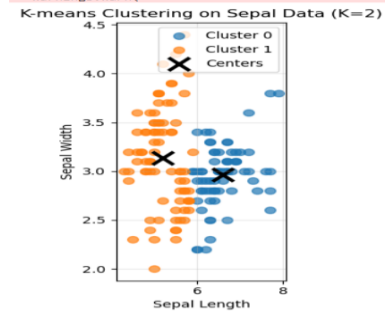
print("\n=== 花瓣数据 K=2 聚类结果 ===")
print("聚类中心:\n", centers_sepal_k2)
print("前10个样本的聚类标签:", labels_sepal_k2[:10])

# 可视化 K=2 的结果
plt.subplot(1, 3, 3)
for i in range(2):
    mask = labels_sepal_k2 == i
    plt.scatter(X_sepal[mask, 0], X_sepal[mask, 1],
                label=f'Cluster {i}', alpha=0.7, s=50)

plt.scatter(centers_sepal_k2[:, 0], centers_sepal_k2[:, 1],
            c='black', marker='x', s=200, linewidth=3, label='Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Clustering on Sepal Data (K=2)')
plt.legend()
plt.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

```
=== 花瓣数据 K=2 聚类结果 ===
聚类中心:
[[6.58985507 2.96666667]
 [5.20740741 3.1345679 ]]
前10个样本的聚类标签: [1 1 1 1 1 1 1 1 1 1]
F:\Anaconda\Lib\site-packages\sklearn\cluster\_kmeans.py:1446: UserWarning: KMeans is known to have a memory leak on Windows with MKL, when there are
ss chunks than available threads. You can avoid it by setting the environment variable OMP_NUM_THREADS=1.
  warnings.warn(
```



四、查考资料，分析该案例，聚类的其他输出，比如：聚类中心（代码）。可视化聚类中心。

```
[43]: # 详细分析聚类中心
print("\n" + "="*50)
print("聚类中心详细分析")
print("="*50)

# 对比不同K值和不同特征集的聚类中心
print("\n1. 花萼数据 K=3 的聚类中心:")
for i, center in enumerate(centers_sepal):
    print(f"    Cluster {i}: 花萼长度={center[0]:.2f}, 花萼宽度={center[1]:.2f}")

print("\n2. 所有数据 K=3 的聚类中心 (前两个维度):")
for i, center in enumerate(centers_all):
    print(f"    Cluster {i}: 花萼长度={center[0]:.2f}, 花萼宽度={center[1]:.2f}")

# 可视化聚类中心 (更清晰的展示)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
# 绘制原始数据点 (用颜色区分真实类别, 以便对比)
plt.scatter(X_sepal[:, 0], X_sepal[:, 1], c=iris.target,
            cmap='viridis', alpha=0.5, s=50)
plt.scatter(centers_sepal[:, 0], centers_sepal[:, 1],
            c='red', marker='*', s=300, edgecolors='black', linewidth=2, label='K-means Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Centers on Sepal Data\n(Colored by True Labels)')
plt.legend()
plt.colorbar(label='True Species')

plt.subplot(1, 2, 2)
# 绘制 K=2 的聚类中心
plt.scatter(X_sepal[:, 0], X_sepal[:, 1], c=labels_sepal_k2,
            cmap='viridis', alpha=0.7, s=50)
plt.scatter(centers_sepal_k2[:, 0], centers_sepal_k2[:, 1],
            c='red', marker='*', s=300, edgecolors='black', linewidth=2, label='K=2 Centers')
plt.xlabel('Sepal Length')
plt.ylabel('Sepal Width')
plt.title('K-means Centers (K=2)\n(Colored by Cluster Labels)')
plt.legend()
plt.colorbar(label='Cluster Label')

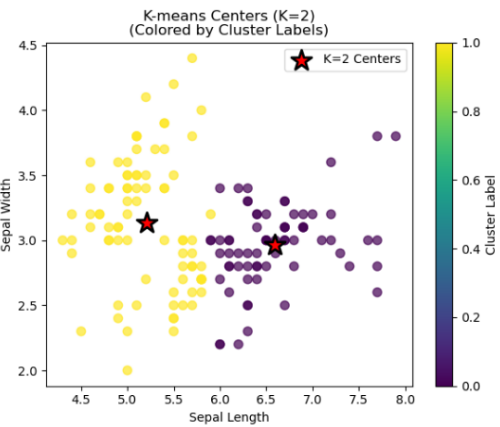
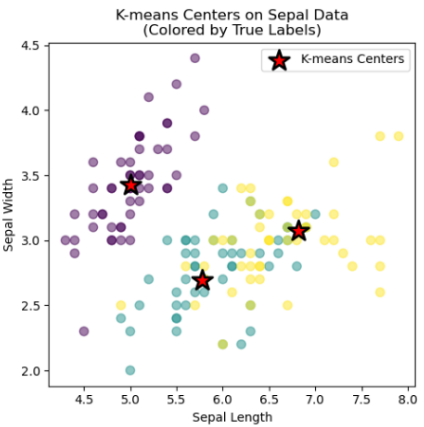
plt.tight_layout()
plt.show()

# 计算并比较不同模型的 Inertia (簇内误差平方和)
print("\n不同模型的 Inertia (簇内误差平方和, 越小越好):")
print(f"花萼数据 K=3: {estimator_sepal.inertia_.2f}")
print(f"所有数据 K=3: {estimator_all.inertia_.2f}")
print(f"花萼数据 K=2: {estimator_sepal_k2.inertia_.2f}")

# 分析聚类效果
print("\n聚类效果分析:")
print("- 当K=3时, 花萼数据将样本分为3组, 但可以看到某些簇重叠较多")
print("- 当K=2时, 算法将样本分为两大类, 左下角一组, 右上角一组")
print("- 使用所有4维数据时, 聚类效果通常比仅用2维更好 (inertia更小)")
print("- 聚类中心表示每个簇的平均特征, 可以用来代表该簇的典型样本")
```

=====
聚类中心详细分析
=====

- 1. 花萼数据 K=3 的聚类中心:
Cluster 0: 花萼长度=6.81, 花萼宽度=3.07
Cluster 1: 花萼长度=5.77, 花萼宽度=2.69
Cluster 2: 花萼长度=5.01, 花萼宽度=3.43
- 2. 所有数据 K=3 的聚类中心 (前两个维度):
Cluster 0: 花萼长度=6.85, 花萼宽度=3.08
Cluster 1: 花萼长度=5.01, 花萼宽度=3.43
Cluster 2: 花萼长度=5.88, 花萼宽度=2.74



不同模型的 Inertia (簇内误差平方和, 越小越好):
花萼数据 K=3: 37.05
所有数据 K=3: 78.86
花萼数据 K=2: 58.21

- 聚类效果分析:
- 当K=3时, 花萼数据将样本分为3组, 但可以看到某些簇重叠较多
 - 当K=2时, 算法将样本分为两大类, 左下角一组, 右上角一组
 - 使用所有4维数据时, 聚类效果通常比仅用2维更好 (inertia更小)
 - 聚类中心表示每个簇的平均特征, 可以用来代表该簇的典型样本